CSS Flexbox Align-Content

Complete Guide: From Basics to Advanced

Contents

Chapter 1

Introduction

1.1 What is align-content?

The `align-content` property aligns flex lines (rows or columns of wrapped items) along the cross axis. It only works when `flex-wrap: wrap` is enabled and items wrap to multiple lines.

Key Point: `align-content` aligns multiple wrapped lines as a group, while `align-items` aligns individual items within a single line.

1.2 align-content vs. align-items

Understanding the difference is crucial:

align-items	**align-content**
Aligns individual items	Aligns wrapped lines
Works with all layouts	Only works with flex-wrap
Aligns items in their line	Aligns entire lines
Controls item positioning	Controls line positioning
Single line or multiple	Only meaningful with multiple lines

1.3 Why Use align-content?

- Control spacing between wrapped lines
- Create balanced multi-line layouts
- Distribute wrapped content evenly
- Perfect for grid-like arrangements
- Handle overflow gracefully
- Professional appearance without manual spacing

1.4 When align-content Applies

```
/* align-content only works with: */
.container {
    display: flex;
    flex-wrap: wrap;  /* Required for align-content to work */
    height: 400px;    /* Needed to see effect */
}

/* Without flex-wrap, align-content has no effect */
.no-wrap {
    display: flex;
    flex-wrap: nowrap;  /* align-content ignored */
}
```

Chapter 2

All align-content Values

2.1 Value Overview

`align-content` has seven main values:

1. `stretch` - Lines stretch to fill container (default)
2. `flex-start` - Lines at start of cross axis
3. `flex-end` - Lines at end of cross axis
4. `center` - Lines centered on cross axis
5. `space-between` - Space between lines
6. `space-around` - Space around lines
7. `space-evenly` - Equal space everywhere

2.2 Comparison Table

Value	**Behavior**
stretch	Lines grow to fill available space
flex-start	Lines packed at start
flex-end	Lines packed at end
center	Lines centered
space-between	Space between lines
space-around	Space around each line
space-evenly	Equal space everywhere

Chapter 3

Core Values

3.1 stretch (Default)

Lines stretch to fill the container along the cross axis.

3.1.1 Syntax

```
.flex-container {
    display: flex;
    flex-wrap: wrap;
    align-content: stretch;  /* Default */
}
```

3.1.2 Characteristics

- Wrapped lines grow to fill container height
- Default behavior
- Creates equal-height lines
- Useful for maximizing space usage
- Professional appearance

3.1.3 Examples

Example 1: Stretched Grid

```
<div class="grid">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
</div>

.grid {
```

```
    display: flex;
    flex-wrap: wrap;
    align-content: stretch;
    height: 300px;
    gap: 10px;
}

.item {
    flex: 1 1 100px;
    background: #3498db;
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
}

/* All rows stretch to fill height evenly */
```

Example 2: Photo Gallery

```
<div class="photo-gallery">
    <img src="photo1.jpg" alt="">
    <img src="photo2.jpg" alt="">
    <img src="photo3.jpg" alt="">
    <!-- More photos -->
</div>

.photo-gallery {
    display: flex;
    flex-wrap: wrap;
    align-content: stretch;
    height: 500px;
    gap: 8px;
}

.photo-gallery img {
    flex: 1 1 150px;
    object-fit: cover;
}

/* Photos stretch to fill height */
```

3.2 flex-start

Lines are packed toward the start of the cross axis.

3.2.1 Syntax

```
.flex-container {
    display: flex;
    flex-wrap: wrap;
    align-content: flex-start;
```

```
}
```

3.2.2 Characteristics

- Lines packed at top (for row) or left (for column)
- Remaining space below/right lines
- Lines maintain natural height
- Useful for top-aligned content
- Common for multi-line layouts

3.2.3 Examples

Example 1: Top-Aligned Grid

```
<div class="grid">
    <div class="card">Card 1</div>
    <div class="card">Card 2</div>
    <div class="card">Card 3</div>
    <div class="card">Card 4</div>
    <div class="card">Card 5</div>
</div>

.grid {
    display: flex;
    flex-wrap: wrap;
    align-content: flex-start;  /* Pack at top */
    height: 400px;
    gap: 15px;
}

.card {
    flex: 1 1 200px;
    background: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

/* Cards packed at top, space below */
```

Example 2: Tag List

```
<div class="tag-list">
    <span class="tag">JavaScript</span>
    <span class="tag">React</span>
    <span class="tag">CSS</span>
    <span class="tag">Flexbox</span>
    <span class="tag">Web</span>
</div>
```

```
.tag-list {
    display: flex;
    flex-wrap: wrap;
    align-content: flex-start;
    height: 300px;
    gap: 8px;
}

.tag {
    background: #3498db;
    color: white;
    padding: 6px 12px;
    border-radius: 12px;
    white-space: nowrap;
}

/* Tags packed at top */
```

3.3 flex-end

Lines are packed toward the end of the cross axis.

3.3.1 Syntax

```
.flex-container {
    display: flex;
    flex-wrap: wrap;
    align-content: flex-end;
}
```

3.3.2 Characteristics

- Lines packed at bottom (for row) or right (for column)
- Remaining space above/left lines
- Lines maintain natural height
- Useful for bottom-aligned content
- Less common but useful

3.3.3 Examples

Example 1: Bottom-Aligned Content

```
<div class="container">
    <div class="item">Item 1</div>
    <div class="item">Item 2</div>
    <div class="item">Item 3</div>
</div>
```

```
.container {
    display: flex;
    flex-wrap: wrap;
    align-content: flex-end;  /* Pack at bottom */
    height: 400px;
    gap: 15px;
}

.item {
    flex: 1 1 150px;
    background: #3498db;
    color: white;
    padding: 20px;
    border-radius: 8px;
}

/* Items packed at bottom */
```

Example 2: Footer Gallery

```
<section class="footer-gallery">
    <img src="img1.jpg" alt="">
    <img src="img2.jpg" alt="">
    <img src="img3.jpg" alt="">
</section>

.footer-gallery {
    display: flex;
    flex-wrap: wrap;
    align-content: flex-end;
    height: 300px;
    gap: 10px;
}

.footer-gallery img {
    flex: 1 1 100px;
    object-fit: cover;
}

/* Gallery at bottom of container */
```

3.4 center

Lines are centered along the cross axis.

3.4.1 Syntax

```
.flex-container {
    display: flex;
    flex-wrap: wrap;
    align-content: center;
}
```

3.4.2 Characteristics

- Lines centered vertically (for row)
- Equal space above and below
- Professional appearance
- Balanced layout
- Works with any height

3.4.3 Examples

Example 1: Centered Grid

```
<div class="showcase">
    <div class="item">Featured 1</div>
    <div class="item">Featured 2</div>
    <div class="item">Featured 3</div>
    <div class="item">Featured 4</div>
</div>

.showcase {
    display: flex;
    flex-wrap: wrap;
    align-content: center;  /* Center vertically */
    justify-content: center;  /* Center horizontally */
    height: 500px;
    gap: 20px;
    padding: 40px;
}

.item {
    flex: 1 1 200px;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    padding: 30px;
    border-radius: 8px;
    text-align: center;
}

/* Items perfectly centered */
```

Example 2: Icon Grid

```
<div class="icon-grid">
    <div class="icon-item">
        <span class="icon "></span>
        <p>Fast</p>
    </div>
    <div class="icon-item">
        <span class="icon "></span>
        <p>Secure</p>
    </div>
    <div class="icon-item">
```

```
        <span class="icon "></span>
        <p>Beautiful</p>
    </div>
</div>

.icon-grid {
    display: flex;
    flex-wrap: wrap;
    align-content: center;
    justify-content: center;
    height: 400px;
    gap: 40px;
}

.icon-item {
    flex: 1 1 120px;
    text-align: center;
}

.icon {
    font-size: 48px;
    display: block;
    margin-bottom: 10px;
}

/* Icons centered both ways */
```

3.5 space-between

Space is distributed between lines.

3.5.1 Syntax

```
.flex-container {
    display: flex;
    flex-wrap: wrap;
    align-content: space-between;
}
```

3.5.2 Characteristics

- First line at start, last line at end
- Remaining space distributed between
- Creates vertical spacing
- Professional appearance
- Great for multi-row layouts

3.5.3 Examples

Example 1: Spaced Grid

```
<div class="grid">
    <div class="row">
        <div class="item">1</div>
        <div class="item">2</div>
        <div class="item">3</div>
    </div>
</div>

.grid {
    display: flex;
    flex-wrap: wrap;
    align-content: space-between;
    height: 400px;
    gap: 15px;
}

.item {
    flex: 1 1 100px;
    background: #3498db;
    color: white;
    padding: 20px;
    border-radius: 4px;
}

/* Rows spaced with space between */
```

Example 2: Multi-Row Product Display

```
<div class="products">
    <div class="product">Product 1</div>
    <div class="product">Product 2</div>
    <div class="product">Product 3</div>
    <div class="product">Product 4</div>
    <div class="product">Product 5</div>
    <div class="product">Product 6</div>
</div>

.products {
    display: flex;
    flex-wrap: wrap;
    align-content: space-between;
    height: 600px;
    gap: 20px;
}

.product {
    flex: 1 1 180px;
    background: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
```

```
    text-align: center;
}

/* Rows well-spaced vertically */
```

3.6 space-around

Space is distributed around each line.

3.6.1 Syntax

```
.flex-container {
    display: flex;
    flex-wrap: wrap;
    align-content: space-around;
}
```

3.6.2 Characteristics

- Equal space around each line
- First and last lines have half-space at edges
- Balanced visual rhythm
- Professional appearance
- Good for centered layouts

3.6.3 Examples

Example 1: Even Spacing

```
<div class="feature-grid">
    <div class="feature">
        <div class="icon "></div>
        <h3>Mobile</h3>
    </div>
    <div class="feature">
        <div class="icon "></div>
        <h3>Desktop</h3>
    </div>
    <div class="feature">
        <div class="icon "></div>
        <h3>Wearable</h3>
    </div>
</div>

.feature-grid {
    display: flex;
    flex-wrap: wrap;
    align-content: space-around;
```

```
    justify-content: center;
    height: 500px;
    gap: 30px;
}

.feature {
    flex: 1 1 150px;
    text-align: center;
}

.icon {
    font-size: 48px;
    margin-bottom: 15px;
}

/* Features evenly spaced */
```

3.7 space-evenly

Equal space is distributed everywhere.

3.7.1 Syntax

```
.flex-container {
    display: flex;
    flex-wrap: wrap;
    align-content: space-evenly;
}
```

3.7.2 Characteristics

- Equal space between all lines
- Equal space at start and end edges
- Most symmetrical distribution
- Perfect balance
- Modern appearance

3.7.3 Examples

Example 1: Perfectly Balanced

```
<div class="gallery">
    <div class="item">Item 1</div>
    <div class="item">Item 2</div>
    <div class="item">Item 3</div>
    <div class="item">Item 4</div>
    <div class="item">Item 5</div>
    <div class="item">Item 6</div>
```

```
</div>

.gallery {
    display: flex;
    flex-wrap: wrap;
    align-content: space-evenly;
    height: 500px;
    gap: 20px;
}

.item {
    flex: 1 1 200px;
    background: white;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

/* Perfect vertical balance */
```

Chapter 4

Practical Applications

4.1 Application 1: Responsive Dashboard

```
<div class="dashboard">
    <div class="card">
        <h3>Card 1</h3>
        <p>Content</p>
    </div>
    <div class="card">
        <h3>Card 2</h3>
        <p>Content</p>
    </div>
    <div class="card">
        <h3>Card 3</h3>
        <p>Content</p>
    </div>
    <div class="card">
        <h3>Card 4</h3>
        <p>Content</p>
    </div>
</div>

.dashboard {
    display: flex;
    flex-wrap: wrap;
    align-content: space-evenly;
    height: 500px;
    gap: 20px;
    padding: 20px;
}

.card {
    flex: 1 1 250px;
    background: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}
```

```
/* Dashboard with balanced card spacing */
```

4.2 Application 2: Product Grid

```
<div class="products">
    <div class="product">
        <img src="product1.jpg" alt="">
        <h3>Product 1</h3>
        <p>$29.99</p>
    </div>
    <!-- More products -->
</div>

.products {
    display: flex;
    flex-wrap: wrap;
    align-content: center;
    justify-content: center;
    min-height: 600px;
    gap: 20px;
    padding: 40px;
}

.product {
    flex: 1 1 200px;
    background: white;
    border-radius: 8px;
    overflow: hidden;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
    text-align: center;
}

/* Products centered in container */
```

4.3 Application 3: Image Gallery

```
<div class="gallery">
    <figure class="gallery-item">
        <img src="image1.jpg" alt="">
        <figcaption>Image 1</figcaption>
    </figure>
    <!-- More images -->
</div>

.gallery {
    display: flex;
    flex-wrap: wrap;
    align-content: space-between;
    height: 600px;
    gap: 10px;
```

```
}

.gallery-item {
    flex: 1 1 150px;
    margin: 0;
}

.gallery-item img {
    width: 100%;
    height: 100%;
    object-fit: cover;
}

/* Gallery with balanced row spacing */
```

Chapter 5

Responsive align-content

5.1 Mobile-First Approach

```
/* Mobile: Top aligned */
.grid {
    display: flex;
    flex-wrap: wrap;
    align-content: flex-start;
    height: auto;  /* Natural height */
}

/* Tablet: Centered */
@media (min-width: 768px) {
    .grid {
        align-content: center;
        height: 500px;
    }
}

/* Desktop: Space-between */
@media (min-width: 1200px) {
    .grid {
        align-content: space-between;
        height: 700px;
    }
}
```

5.2 Height-Based Responsiveness

```
.responsive-grid {
    display: flex;
    flex-wrap: wrap;
    align-content: space-evenly;
    height: clamp(400px, 80vh, 800px);
    gap: 15px;
}
```

```
.item {
    flex: 1 1 150px;
}

/* Height adjusts responsively */
```

Chapter 6

Combining align-content with Other Properties

6.1 align-content with align-items

Use both for complete 2D control:

```
.container {
    display: flex;
    flex-wrap: wrap;
    align-content: space-between;  /* Lines spacing */
    align-items: center;            /* Items in line */
    height: 500px;
}

/* Lines spaced, items centered in each line */
```

6.2 align-content with justify-content

```
.grid {
    display: flex;
    flex-wrap: wrap;
    justify-content: space-around;    /* Horizontal spacing */
    align-content: space-evenly;      /* Vertical spacing */
    height: 600px;
    gap: 20px;
}

/* Perfect 2D spacing */
```

6.3 align-content with gap

The `gap` property works with `align-content`:

```
.container {
    display: flex;
```

```
    flex-wrap: wrap;
    align-content: center;
    gap: 20px;  /* Fixed gap between items */
}

/* Gap provides consistent spacing */
```

Chapter 7

Troubleshooting

7.1 Issue 1: align-content Not Working

Problem: `align-content` has no effect.

Solution: Ensure `flex-wrap: wrap` and container height are set.

```
/* WRONG: Missing flex-wrap or height */
.container {
    display: flex;
    align-content: center;  /* No effect */
}

/* CORRECT: Add flex-wrap and height */
.container {
    display: flex;
    flex-wrap: wrap;
    align-content: center;
    height: 400px;  /* Now it works */
}
```

7.2 Issue 2: Conflicting with align-items

Problem: `align-items` and `align-content` conflict.

Solution: Remember they work at different levels. Use both intentionally.

```
.container {
    display: flex;
    flex-wrap: wrap;
    align-items: center;      /* Items within line */
    align-content: flex-start;  /* Lines in container */
}

/* Both work together, not in conflict */
```

7.3 Issue 3: Uneven Line Heights

Problem: Lines have different heights.

Solution: Use `align-content: stretch` to equalize.

```
.container {
    display: flex;
    flex-wrap: wrap;
    align-content: stretch;  /* Lines grow equally */
    height: 400px;
}
```

Chapter 8

Best Practices

8.1 When to Use Each Value

- **stretch:** Maximize space, equal-height layouts
- **flex-start:** Top-aligned multi-row content
- **flex-end:** Bottom-aligned content
- **center:** Centered multi-line layouts
- **space-between:** First/last line at edges
- **space-around:** Evenly distributed with edges
- **space-evenly:** Perfect symmetry

8.2 Common Patterns

```
/* Centered multi-line layout */
.centered {
    display: flex;
    flex-wrap: wrap;
    align-content: center;
    justify-content: center;
    height: 500px;
}

/* Spaced rows */
.spaced {
    display: flex;
    flex-wrap: wrap;
    align-content: space-between;
    height: 600px;
}

/* Stretched to fill */
.filled {
    display: flex;
```

```
    flex-wrap: wrap;
    align-content: stretch;
    height: 400px;
}

/* Perfectly balanced */
.balanced {
    display: flex;
    flex-wrap: wrap;
    align-content: space-evenly;
    justify-content: center;
    height: 500px;
}
```

8.3 Accessibility Tips

- Maintain logical reading order
- Don't hide important content
- Ensure sufficient spacing for touch targets
- Test keyboard navigation
- Consider screen reader experience

Chapter 9

Summary: Key Takeaways

9.1 The Seven Values

Value	Best For
stretch	Maximizing space
flex-start	Top alignment
flex-end	Bottom alignment
center	Centered content
space-between	Edge alignment
space-around	Balanced spacing
space-evenly	Perfect symmetry

9.2 Key Rules

1. Only works with `flex-wrap: wrap`
2. Requires container height to see effect
3. Aligns wrapped lines, not individual items
4. Works on cross axis
5. Different from `align-items`
6. Combine with `align-items` for complete control
7. Most common: center, space-evenly, stretch
8. Responsive with media queries
9. Test at different container heights
10. Use with `gap` for spacing

9.3 Quick Reference

```css
/* Centered wrapped lines */
.centered {
    display: flex;
    flex-wrap: wrap;
    align-content: center;
    height: 400px;
}

/* Space between lines */
.spaced {
    display: flex;
    flex-wrap: wrap;
    align-content: space-between;
    height: 500px;
}

/* Evenly distributed */
.even {
    display: flex;
    flex-wrap: wrap;
    align-content: space-evenly;
    height: 600px;
}

/* Stretched lines */
.stretched {
    display: flex;
    flex-wrap: wrap;
    align-content: stretch;
    height: 400px;
}

/* Perfect 2D control */
.perfect {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    align-items: center;
    align-content: center;
    height: 500px;
}
```

9.4 align-content vs. align-items

- **align-items:** Aligns items in their line
- **align-content:** Aligns wrapped lines themselves
- Only one line → `align-items` visible
- Multiple lines → Both have effect
- Use together for complete 2D layout

9.5 When flex-wrap Required

With flex-wrap	**Without flex-wrap**
align-content works	align-content has no effect
Multiple lines possible	Single line only
Lines can wrap	Items shrink to fit
align-content relevant	align-items only